

Project Report

Design a 24-Bit Custom CPU in Logisim

Objectives

- Design and implement a 24-bit CPU in Logisim following Von Neumann architecture.
- Implement a instruction with a 4-bit opcode, supporting arithmetic, logic, memory, branch, shift, and multiply operations.
- Build a microprogrammed control unit using a ROM lookup table to generate control signals.
- Demonstrate correct fetch-execute cycle operation through simulation with a memory-mapped program.
- Incorporate Booth's multiplication algorithm as a hardware unit for efficient 2's complement multiplication.

Introduction

This project presents the design and simulation of a custom 24-bit central processing unit (CPU). The processor is built on the Von Neumann architecture, where a single unified memory holds both the program instructions and the data on which the program operates. Each memory word is 24 bits wide, and memory is accessed through address and buffer registers that form the interface between the processor and memory.

The instruction word is 24 bits long. The upper 4 bits carry the operation code, which tells the control unit which instruction to execute. The next 8 bits carry the memory address of the operand. The remaining 12 bits are unused during an instruction word fetch but carry the full 24-bit value when a data word is read from memory during the execute phase. This split interpretation of the memory word is a fundamental characteristic of Von Neumann design.

The processor contains a Program Counter (PC), a Memory Address Register (MAR), a Memory Buffer Register (MBR), an Accumulator (AC), an Instruction Register (IR), and an Arithmetic Logic Unit (ALU). A microprogrammed control unit drives all register transfers by reading a sequence of microoperations from a ROM for each instruction. The design supports 14 instructions covering arithmetic, logic, memory access, conditional and unconditional branching, circular rotation, and hardware multiplication using Booth's algorithm.

Instruction Set and Functions

The processor supports 14 instructions, each identified by a unique 4-bit operation code. The table below lists every instruction along with its register-transfer level operation and a brief description.

Opcode	Mnemonic	Operation	Description
0 (0000)	AND	$A \leftarrow A \wedge M$	Bitwise AND of Accumulator with memory word
1 (0001)	OR	$A \leftarrow A \vee M$	Bitwise OR of Accumulator with memory word
2 (0010)	STO	$M \leftarrow A$	Store Accumulator into memory
3 (0011)	ADD	$A \leftarrow A + M$	Add memory word to Accumulator
4 (0100)	SUB	$A \leftarrow A - M$	Subtract memory word from Accumulator

5 (0101)	BUN	$PC \leftarrow m$	Branch unconditionally to address m
6 (0110)	LDA	$A \leftarrow M$	Load memory word into Accumulator
7 (0111)	INC	$A \leftarrow A + 1$	Increment Accumulator by 1
8 (1000)	DEC	$A \leftarrow A - 1$	Decrement Accumulator by 1
9 (1001)	ROL	$A \leftarrow \text{rot_L}(A)$	Rotate Accumulator left (circular)
A (1010)	ROR	$A \leftarrow \text{rot_R}(A)$	Rotate Accumulator right (circular)
B (1011)	JN	if $A < 0$: $PC \leftarrow m$	Jump to m if Accumulator is negative
C (1100)	JNE	if $A \neq 0$: $PC \leftarrow m$	Jump to m if Accumulator is not zero
D (1101)	MUL	$\text{Result} \leftarrow A \times M$	Multiply Accumulator by memory word (Booth's algorithm)

Fetch Cycle

Every instruction begins with the same three-step fetch sequence before any instruction-specific action takes place:

- $MAR \leftarrow PC$ — the current program counter value is copied into the memory address register.
- $MBR \leftarrow M$, $PC \leftarrow PC + 1$ — the instruction word is read from memory into the buffer register, and the program counter is incremented to point to the next instruction.
- $IR \leftarrow MBR[OP]$ — the upper 4 bits of the buffer register (the operation code) are transferred into the instruction register.

Once the opcode is in the instruction register, the control unit decodes it and branches to the corresponding execute routine stored in the control ROM.

Execution Phase

For memory-reference instructions such as AND, OR, ADD, SUB, STO, and LDA, the execute phase begins by loading the address field of the instruction from the buffer register into MAR. A second memory read then places the data operand into MBR, after which the specified operation is carried out between the accumulator and the operand.

Register-only instructions — INC, DEC, ROL, and ROR — require no second memory access. They operate directly on the accumulator and complete in a single execution step. Branch instructions read the address field from the buffer register and either load it unconditionally into the program counter (BUN) or do so conditionally based on the accumulator state (JN, JNE). The multiply instruction (MUL) passes control to the Booth hardware unit.

Instruction Descriptions

- AND / OR: Perform a bitwise logical operation between the accumulator and the memory operand. The result replaces the previous accumulator value.
- ADD / SUB: Perform 24-bit addition or subtraction using 2's complement representation.
- LDA / STO: LDA copies a memory word into the accumulator. STO copies the accumulator into a memory location. These two instructions handle all data movement between the processor and memory.
- INC / DEC: Add or subtract one from the accumulator without any memory access.
- ROL / ROR: Rotate the accumulator one bit position to the left or right in a circular manner. The bit shifted out from one end is inserted at the other end.

- BUN: Transfer control unconditionally to the instruction at the address given in the instruction word.
- JN: Transfer control to the given address only if the accumulator holds a negative value, indicated by the sign bit being 1.
- JNN: Transfer control to the given address only if the accumulator doesn't hold a negative value.
- MUL: Invoke the Booth hardware multiplier to compute the product of the accumulator and a memory operand.

CPU and Control Unit Design

Register Architecture

The processor is a single-accumulator machine, meaning all arithmetic and logic operations use the accumulator as both the source of one operand and the destination of the result. The registers are:

- PC (Program Counter): Stores the address of the next instruction. It can be incremented by one or loaded with a new address for branching.
- MAR (Memory Address Register): Holds the address sent to memory for every read or write operation.
- MBR (Memory Buffer Register): Acts as the gateway between the processor and memory. It receives instruction words during fetch and data words during execute, and it holds values to be written back to memory.
- IR (Instruction Register): Stores the 4-bit opcode extracted from the upper bits of the MBR after fetch. It preserves the opcode even after the MBR is overwritten by a data read.
- AC (Accumulator): The central 24-bit working register. Every arithmetic, logic, and shift result is stored here.

Arithmetic Logic Unit

The ALU receives its two inputs from the accumulator and the memory buffer register. It supports addition, subtraction, bitwise AND, bitwise OR, increment, decrement, and circular rotation. The operation performed is selected by control signals issued by the control unit. The ALU also produces two status outputs: a negative flag that reflects the sign bit of the accumulator, and a zero flag that becomes active when the accumulator holds the value zero. These flags are used by the conditional branch instructions.

Microprogrammed Control Unit

The control unit is microprogrammed. It contains a read-only memory (ROM) that stores the complete set of microoperations for every instruction. A control address register indexes into this ROM. At the end of each fetch, the opcode in the instruction register is used to calculate the starting address of the corresponding execute routine in the ROM, and control jumps to that address.

Each ROM word specifies one microoperation and encodes the address of the next microinstruction. The outputs of the ROM drive the control signals throughout the processor: signals to transfer the program counter to MAR, to increment the program counter, to load data from memory into the MBR, to load the accumulator, to write data back to memory, to enable the multiplier, and others. The control unit also receives the negative flag, zero flag, and a busy signal from the Booth multiplier, all of which influence the control flow for conditional and multiply instructions.

Booth's Multiplication Unit

Opcode D (1101) is assigned to the multiply instruction. When this instruction is executed, the control unit activates the Booth hardware multiplier. The multiplier takes two 24-bit signed operands in 2's complement form and computes their product using the Booth algorithm, which handles both positive and negative numbers correctly without special cases.

The Booth unit is a sequential circuit. Unlike the rest of the processor, which completes each microoperation in a single clock cycle, the Booth multiplier requires multiple clock cycles to finish its computation. The number of cycles depends on the bit width of the operands. During this time, the multiplier asserts a busy signal that holds the control unit in a wait state, preventing it from advancing to the next instruction until the multiplication is complete.

When the Booth unit finishes, it produces a 48-bit result — twice the width of a single operand — because the product of two 24-bit numbers can require up to 48 bits. This result is split across two dedicated output registers: MUL_OUTPUT_1, which holds the lower half of the product, and MUL_OUTPUT_2, which holds the upper half.

Discussion

The CPU operates correctly across all 13 instructions. The fetch cycle properly separates address transfer, memory read, and opcode extraction, ensuring the instruction register retains the opcode throughout the execute phase.

Limitations

MUL_OUTPUT not on the data bus. The Booth multiplier produces the correct 48-bit product but the result registers are isolated from the main bus, so no instruction can move the result into the accumulator or memory. Fixing this requires connecting MUL_OUTPUT_1 and MUL_OUTPUT_2 to the accumulator input and adding the corresponding control signals.

256-location address limit. Only 8 bits are available for the address field, capping directly addressable memory at 256 locations. Wider addressing would require a wider instruction word or indirect addressing.

No interrupts. The processor runs instructions strictly in order and cannot respond to external events. An interrupt mechanism would need an input line, a way to save the program counter, and a return instruction.

Single accumulator. All operations pass through one register, forcing frequent loads and stores for complex expressions. A small register file would reduce this overhead.

No pipeline. Each instruction completes fully before the next begins. Even a basic two-stage pipeline would significantly improve throughput.

7. Conclusion

This project successfully designed, implemented, and simulated a 24-bit CPU. The processor follows Von Neumann architecture with a microprogrammed control unit and a 14-instruction set that covers arithmetic, logic, memory access, conditional and unconditional branching, circular rotation, and hardware multiplication.

The fetch-execute cycle was verified to work correctly for all instruction types. The control unit decoded each opcode accurately and issued the correct sequence of microoperations in every case. The ALU status flags correctly controlled conditional branching, and the rotate mechanism worked without error using the carry flip-flop as a wrap-around buffer.

Beyond the multiplication limitation, the design is complete and functional. Future improvements could include connecting the multiply output registers to the accumulator data path, adding an interrupt mechanism, expanding the address space beyond 8 bits, implementing a hardware divide unit, and introducing instruction pipelining to increase throughput.